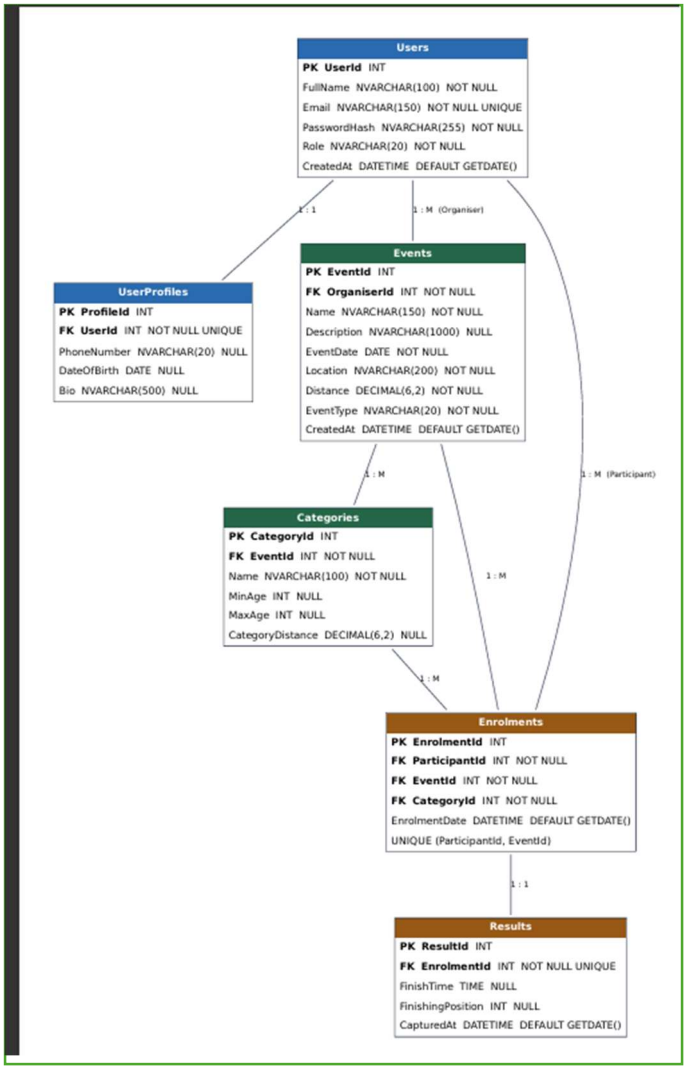
PROGRAMMING 2B (PROG6212)

ST10475585 MUDAU OMPHULUSA IMANI

POE PART 2

ERD


Users
PK UserId INT
FullName NVARCHAR(100) NOT NULL
Email NVARCHAR(150) NOT NULL UNIQUE
PasswordHash NVARCHAR(255) NOT NULL
Role NVARCHAR(20) NOT NULL
CreatedAt DATETIME DEFAULT GETDATE()
1 : 1
1 : M (Organiser)
UserProfiles
PK ProfileId INT
FK UserId INT NOT NULL UNIQUE
PhoneNumber NVARCHAR(20) NULL
DateOfBirth DATE NULL
Bio NVARCHAR(500) NULL
Events
PK EventId INT
FK OrganiserId INT NOT NULL
Name NVARCHAR(150) NOT NULL
Description NVARCHAR(1000) NULL
EventDate DATE NOT NULL
Location NVARCHAR(200) NOT NULL
Distance DECIMAL(6,2) NOT NULL
EventType NVARCHAR(20) NOT NULL
CreatedAt DATETIME DEFAULT GETDATE()
1 : M
1 : M (Participant)
Categories
PK CategoryId INT
FK EventId INT NOT NULL
Name NVARCHAR(100) NOT NULL
MinAge INT NULL
MaxAge INT NULL
CategoryDistance DECIMAL(6,2) NULL
1 : M
1 : M
Enrolments
PK EnrolmentId INT
FK ParticipantId INT NOT NULL
FK EventId INT NOT NULL
FK CategoryId INT NOT NULL
EnrolmentDate DATETIME DEFAULT GETDATE()
UNIQUE (ParticipantId, EventId)
1 : 1
Results
PK ResultId INT
FK EnrolmentId INT NOT NULL UNIQUE
FinishTime TIME NULL
FinishingPosition INT NULL
CapturedAt DATETIME DEFAULT GETDATE()

Sql

```sql
/* ============================================================
   RaceDay Database Schema
   SQL Server Management Studio (SSMS)
   Matches erd.png exactly - 6 entities, all PKs/FKs/constraints
   ============================================================ */

IF DB_ID('RaceDayDB') IS NULL
BEGIN
    CREATE DATABASE RaceDayDB;
END
GO

USE RaceDayDB;
GO

/* Drop tables in FK-safe order if re-running the script */
IF OBJECT_ID('dbo.Results', 'U') IS NOT NULL DROP TABLE dbo.Results;
IF OBJECT_ID('dbo.Enrolments', 'U') IS NOT NULL DROP TABLE dbo.Enrolments;
IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL DROP TABLE dbo.Categories;
IF OBJECT_ID('dbo.Events', 'U') IS NOT NULL DROP TABLE dbo.Events;
IF OBJECT_ID('dbo.UserProfiles', 'U') IS NOT NULL DROP TABLE dbo.UserProfiles;
IF OBJECT_ID('dbo.Users', 'U') IS NOT NULL DROP TABLE dbo.Users;
GO

/* ============================================================
   1. Users
   ============================================================ */
CREATE TABLE dbo.Users (
    UserId          INT IDENTITY(1,1)       NOT NULL,
```

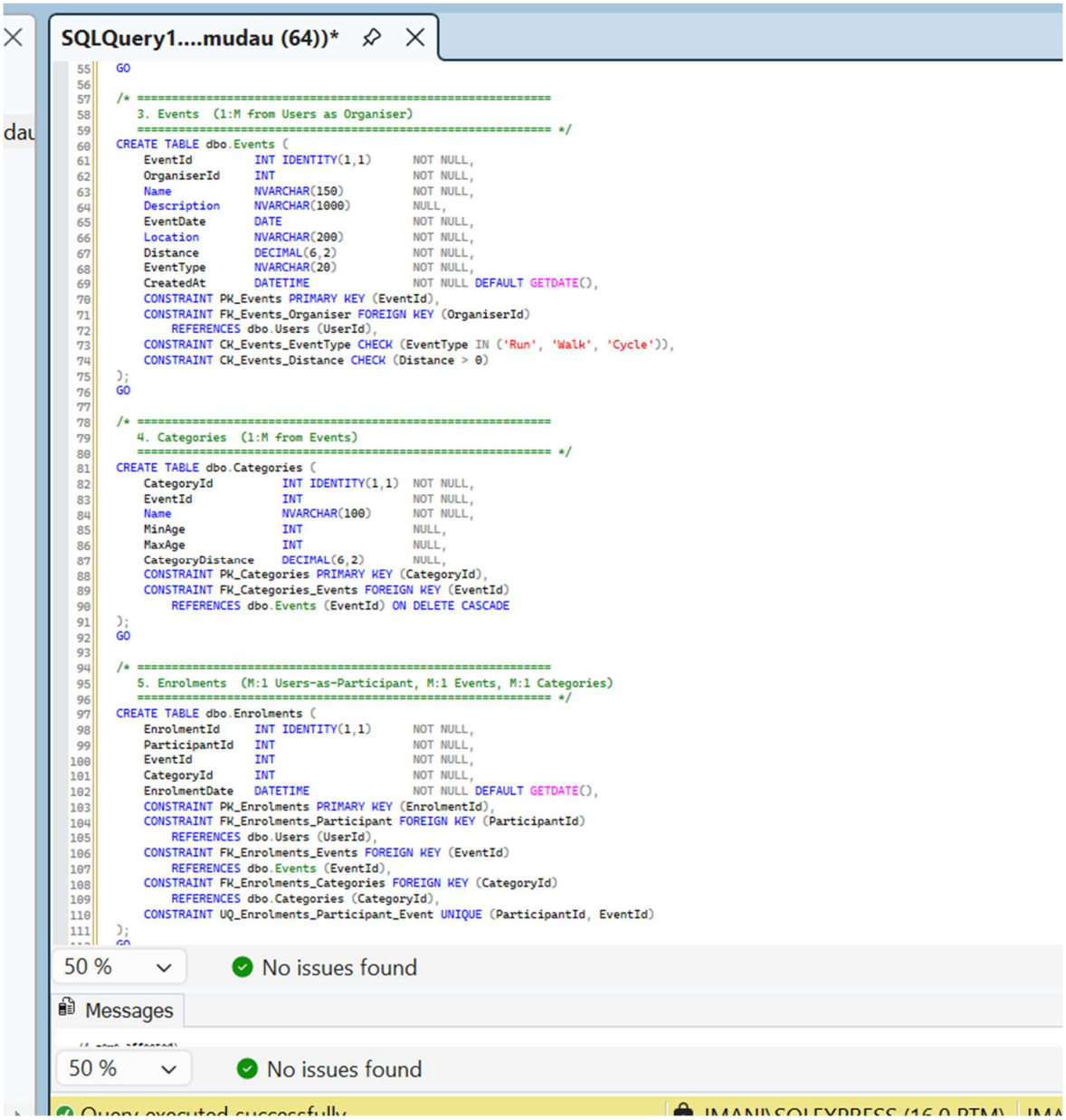
No issues found

SQLQuery1....mudau (64))*

```sql
CREATE TABLE dbo.Users (
    UserId          INT IDENTITY(1,1)       NOT NULL,
    FullName        NVARCHAR(100)           NOT NULL,
    Email           NVARCHAR(150)           NOT NULL,
    PasswordHash    NVARCHAR(255)           NOT NULL,
    Role            NVARCHAR(20)            NOT NULL DEFAULT 'Participant',
    CreatedAt       DATETIME                NOT NULL DEFAULT GETDATE(),
    CONSTRAINT PK_Users PRIMARY KEY (UserId),
    CONSTRAINT UQ_Users_Email UNIQUE (Email),
    CONSTRAINT CK_Users_Role CHECK (Role IN ('Organiser', 'Participant'))
);
GO

/* ============================================================
   2. UserProfiles  (1:1 with Users)
   ============================================================ */
CREATE TABLE dbo.UserProfiles (
    ProfileId       INT IDENTITY(1,1)       NOT NULL,
    UserId          INT                     NOT NULL,
    PhoneNumber     NVARCHAR(20)            NULL,
    DateOfBirth     DATE                    NULL,
    Bio             NVARCHAR(500)           NULL,
    CONSTRAINT PK_UserProfiles PRIMARY KEY (ProfileId),
    CONSTRAINT UQ_UserProfiles_UserId UNIQUE (UserId),
    CONSTRAINT FK_UserProfiles_Users FOREIGN KEY (UserId)
        REFERENCES dbo.Users (UserId) ON DELETE CASCADE
);
GO
```

100 % No issues found Ln: 1,


SQLQuery1....mudau (64))*
GO
/* ==
3. Events (1:M from Users as Organiser)
== */
CREATE TABLE dbo.Events (
EventId INT IDENTITY(1,1) NOT NULL,
OrganiserId INT NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(1000) NULL,
EventDate DATE NOT NULL,
Location NVARCHAR(200) NOT NULL,
Distance DECIMAL(6,2) NOT NULL,
EventType NVARCHAR(20) NOT NULL,
CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
CONSTRAINT PK_Events PRIMARY KEY (EventId),
CONSTRAINT FK_Events_Organiser FOREIGN KEY (OrganiserId)
REFERENCES dbo.Users (UserId),
CONSTRAINT CK_Events_EventType CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
CONSTRAINT CK_Events_Distance CHECK (Distance > 0)
);
GO
/* ==
4. Categories (1:M from Events)
== */
CREATE TABLE dbo.Categories (
CategoryId INT IDENTITY(1,1) NOT NULL,
EventId INT NOT NULL,
Name NVARCHAR(100) NOT NULL,
MinAge INT NULL,
MaxAge INT NULL,
CategoryDistance DECIMAL(6,2) NULL,
CONSTRAINT PK_Categories PRIMARY KEY (CategoryId),
CONSTRAINT FK_Categories_Events FOREIGN KEY (EventId)
REFERENCES dbo.Events (EventId) ON DELETE CASCADE
);
GO
/* ==
5. Enrolments (M:1 Users-as-Participant, M:1 Events, M:1 Categories)
== */
CREATE TABLE dbo.Enrolments (
EnrolmentId INT IDENTITY(1,1) NOT NULL,
ParticipantId INT NOT NULL,
EventId INT NOT NULL,
CategoryId INT NOT NULL,
EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),
CONSTRAINT PK_Enrolments PRIMARY KEY (EnrolmentId),
CONSTRAINT FK_Enrolments_Participant FOREIGN KEY (ParticipantId)
REFERENCES dbo.Users (UserId),
CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventId)
REFERENCES dbo.Events (EventId),
CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId)
REFERENCES dbo.Categories (CategoryId),
CONSTRAINT UQ_Enrolments_Participant_Event UNIQUE (ParticipantId, EventId)
);
GO
50 %
No issues found
Messages
50 %
No issues found

```
GO

/* ============================================================
   6. Results  (1:1 with Enrolments)
   ============================================================ */
CREATE TABLE dbo.Results (
    ResultId            INT IDENTITY(1,1)  NOT NULL,
    EnrolmentId         INT                NOT NULL,
    FinishTime          TIME               NULL,
    FinishingPosition   INT                NULL,
    CapturedAt          DATETIME           NOT NULL DEFAULT GETDATE(),
    CONSTRAINT PK_Results PRIMARY KEY (ResultId),
    CONSTRAINT UQ_Results_EnrolmentId UNIQUE (EnrolmentId),
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId)
        REFERENCES dbo.Enrolments (EnrolmentId) ON DELETE CASCADE
);
GO

/* ============================================================
   SEED DATA
   Note: PasswordHash values below are placeholder bcrypt-style
   strings for seed/demo purposes only - real hashes are produced
   by the API's registration endpoint, not typed here directly.
   ============================================================ */

-- 2 Organisers + 2 Participants
INSERT INTO dbo.Users (FullName, Email, PasswordHash, Role) VALUES
('Sarah Nkosi',     'sarah.nkosi@raceday.co.za',   '$2a$11$examplehash.organiser.one', 'Organiser'),
('David Botha',     'david.botha@raceday.co.za',   '$2a$11$examplehash.organiser.two', 'Organiser'),
('Lindiwe Dube',    'lindiwe.dube@example.com',    '$2a$11$examplehash.participant.a', 'Participant'),
('Michael Reddy',   'michael.reddy@example.com',   '$2a$11$examplehash.participant.b', 'Participant');
GO

INSERT INTO dbo.UserProfiles (UserId, PhoneNumber, DateOfBirth, Bio) VALUES
(1, '0821234567', '1985-03-12', 'Event organiser specialising in road running.'),
(2, '0837654321', '1979-11-02', 'Organiser for cycling and multi-sport events.'),
(3, '0715558899', '1998-06-21', 'Keen 10km and half-marathon runner.'),
(4, '0723334455', '1990-01-15', 'Weekend cyclist training for first century ride.');
GO

-- 3 Events (owned by the 2 organisers)
INSERT INTO dbo.Events (OrganiserId, Name, Description, EventDate, Location, Distance, EventType) VALUES
(1, 'Johannesburg City Run',   'Annual road running event through the city centre.', '2026-09-12', 'Johannesburg, Gauteng', 21.10, 'Run'),
(1, 'Sunrise Fun Walk',        'Community walk supporting local charities.',         '2026-08-30', 'Pretoria, Gauteng',     5.00, 'Walk'),
(2, 'Highveld Cycle Challenge','Road cycling race across the Highveld region.',      '2026-10-04', 'Bronkhorstspruit, Gauteng', 90.00, 'Cycle');
GO

-- Categories per event
INSERT INTO dbo.Categories (EventId, Name, MinAge, MaxAge, CategoryDistance) VALUES
(1, 'Senior 21km',   20, 39, 21.10),
(1, 'Veteran 21km',  40, 99, 21.10),
(1, 'Under 20',      13, 19, 21.10),
(2, '5km Walk',       0, 99, 5.00),
(3, '90km Road Race', 18, 99, 90.00),
(3, '45km Fun Ride',  14, 99, 45.00);
GO
```

50 % ✓ No issues found

Messages

SQLQuery1....mudau (64))*

```
('Sarah Nkosi',     'sarah.nkosi@raceday.co.za',   '$2a$11$examplehash.organiser.one', 'Organiser'),
('David Botha',     'david.botha@raceday.co.za',   '$2a$11$examplehash.organiser.two', 'Organiser'),
('Lindiwe Dube',    'lindiwe.dube@example.com',    '$2a$11$examplehash.participant.a', 'Participant'),
('Michael Reddy',   'michael.reddy@example.com',   '$2a$11$examplehash.participant.b', 'Participant');
GO

INSERT INTO dbo.UserProfiles (UserId, PhoneNumber, DateOfBirth, Bio) VALUES
(1, '0821234567', '1985-03-12', 'Event organiser specialising in road running.'),
(2, '0837654321', '1979-11-02', 'Organiser for cycling and multi-sport events.'),
(3, '07155588899', '1998-06-21', 'Keen 10km and half-marathon runner.'),
(4, '07233344455', '1990-01-15', 'Weekend cyclist training for first century ride.');
GO

-- 3 Events (owned by the 2 organisers)
INSERT INTO dbo.Events (OrganiserId, Name, Description, EventDate, Location, Distance, EventType) VALUES
(1, 'Johannesburg City Run',    'Annual road running event through the city centre.', '2026-09-12', 'Johannesburg, Gauteng', 21.10, 'Run'),
(1, 'Sunrise Fun Walk',         'Community walk supporting local charities.',         '2026-08-30', 'Pretoria, Gauteng',     5.00, 'Walk'),
(2, 'Highveld Cycle Challenge','Road cycling race across the Highveld region.',      '2026-10-04', 'Bronkhorstspruit, Gauteng', 90.00, 'Cycle');
GO

-- Categories per event
INSERT INTO dbo.Categories (EventId, Name, MinAge, MaxAge, CategoryDistance) VALUES
(1, 'Senior 21km',    20, 39, 21.10),
(1, 'Veteran 21km',   40, 99, 21.10),
(1, 'Under 20',       13, 19, 21.10),
(2, '5km Walk',        0, 99, 5.00),
(3, '90km Road Race', 18, 99, 90.00),
(3, '45km Fun Ride',  14, 99, 45.00);
GO

-- Sample enrolments
INSERT INTO dbo.Enrolments (ParticipantId, EventId, CategoryId) VALUES
(3, 1, 1),   -- Lindiwe enters Johannesburg City Run, Senior 21km
(4, 1, 2),   -- Michael enters Johannesburg City Run, Veteran 21km
(3, 3, 5),   -- Lindiwe enters Highveld Cycle Challenge, 90km Road Race
(4, 2, 4);   -- Michael enters Sunrise Fun Walk, 5km Walk
GO

-- Sample results for a couple of completed enrolments
INSERT INTO dbo.Results (EnrolmentId, FinishTime, FinishingPosition) VALUES
(1, '01:45:32', 12),
(2, '01:58:10', 27);
GO
```

100 % No issues found

Messages | Client Statistics

	Trial 1	Average
Query Profile Statistics		
Number of INSERT, DELETE and UPDATE statements	6	6.0000
Rows affected by INSERT, DELETE, or UPDATE stateme...	23	23.0000
Number of SELECT statements	0	0.0000
Rows returned by SELECT statements	0	0.0000
Number of transactions	6	6.0000
Network Statistics		
Number of server roundtrips	15	15.0000
TDS packets sent from client	15	15.0000
TDS packets received from server	15	15.0000
Bytes sent from client	17278	17278.0000
Bytes received from server	735	735.0000
Time Statistics		
Client processing time	4	4.0000
Total execution time	328	328.0000
Wait time on server replies	324	324.0000

Query executed successfully. | IMANI\SQLEXPRESS (16.0 RTM) | IMANI\mudau (64) | RaceDayDB | 00

API Endpoint Plan

This plan covers Authentication, User Profile, Events, Categories, Event Enrolments, and Results, as required by the Part 2 Functional Requirements. Role-based access is enforced on every non-public endpoint; session management maintains the authenticated user's identity and role for subsequent requests.

Authentication

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as either an Organiser or a Participant.	None (public)	{ fullName, email, password, role }	201 Created - user record created (no password returned) 400 Bad Request - invalid/missing fields 409 Conflict - email already registered
POST	/api/auth/login	Authenticates a user and establishes their session, recording identity and role.	None (public)	{ email, password }	200 OK - session established, user id and role returned 401 Unauthorized - invalid credentials
POST	/api/auth/logout	Ends the current authenticated session.	Any (logged in)	None	200 OK - session ended 401 Unauthorized - no active session

User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me	Returns the logged-in user's own account and profile details.	Any (logged in)	None	200 OK - user + profile data 401 Unauthorized - not logged in
PUT	/api/users/me	Updates the logged-in user's own profile information.	Any (logged in)	{ fullName, phoneNumber, dateOfBirth, bio }	200 OK - updated profile returned 400 Bad Request - invalid fields 401 Unauthorized - not logged in

Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Lists all events. Visible to both roles for browsing.	Any (logged in)	None	200 OK - array of events
GET	/api/events/{id}	Retrieves the details of a single event.	Any (logged in)	None	200 OK - event details 404 Not Found - event does not exist
POST	/api/events	Creates a new event owned by the logged-in Organiser.	Organiser	{ name, description, eventDate, location, distance, eventType }	201 Created - new event record 400 Bad Request - invalid fields 403 Forbidden - caller is a Participant
PUT	/api/events/{id}	Updates an event owned by the logged-in Organiser.	Organiser	{ name, description, eventDate, location, distance, eventType }	200 OK - updated event 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist
DELETE	/api/events/{id}	Deletes an event owned by the logged-in Organiser.	Organiser	None	204 No Content - event deleted 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist

Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Lists all categories available for a given event.	Any (logged in)	None	200 OK - array of categories 404 Not Found - event does not exist
POST	/api/events/{eventId}/categories	Defines a new age or distance	Organiser	{ name, minAge, maxAge,	201 Created - new category record 403

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		category for an event owned by the logged-in Organiser.		categoryDistance }	Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist
PUT	/api/categories/{id}	Updates a category belonging to an event owned by the logged-in Organiser.	Organiser	{ name, minAge, maxAge, categoryDistance }	200 OK - updated category 403 Forbidden - not the owning organiser or a Participant 404 Not Found - category does not exist
DELETE	/api/categories/{id}	Deletes a category belonging to an event owned by the logged-in Organiser.	Organiser	None	204 No Content - category deleted 403 Forbidden - not the owning organiser or a Participant 404 Not Found - category does not exist

Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/events/{eventId}/enrolments	Enters the logged-in Participant into the event under a selected category.	Participant	{ categoryId }	201 Created - enrolment record linking participant, event and category 400 Bad Request - category does not belong to the event 403 Forbidden - caller

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					is an Organiser 404 Not Found - event or category does not exist 409 Conflict - participant already enrolled in this event
GET	/api/events/{eventId}/enrolments	Lists all enrolments for an event owned by the logged-in Organiser.	Organiser	None	200 OK - array of enrolments with participant and category details 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist
GET	/api/users/me/enrolments	Lists the logged-in Participant's own enrolments across all events.	Participant	None	200 OK - array of the participant's enrolments

Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{enrolmentId}/results	Captures the finish time and finishing position for a participant's	Organiser	{ finishTime, finishingPosition }	201 Created - result record created 403 Forbidden - not the event's organiser or a Participant 404 Not Found -

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
		enrolment, by the event's Organiser.			enrolment does not exist 409 Conflict - result already captured for this enrolment
PUT	/api/enrolments/{enrolmentId}/results	Updates a previously captured result for a participant's enrolment.	Organiser	{ finishTime, finishingPosition }	200 OK - updated result 403 Forbidden - not the event's organiser or a Participant 404 Not Found - result does not exist
GET	/api/users/me/results	Lists the logged-in Participant's own results across all events entered.	Participant	None	200 OK - array of results with event and category context
GET	/api/events/{eventId}/results	Lists all results for an event owned by the logged-in Organiser.	Organiser	None	200 OK - array of results for the event 403 Forbidden - not the event's organiser or a Participant 404 Not Found - event does not exist

RaceDay

RaceDay is an event management system for run, walk and cycle events. Organisers create events and categories and capture results; Participants enrol in events and view their own results.

- **Part 1** - planning documentation (ERD, SQL script, API endpoint plan) - in /docs
- **Part 2** - ASP.NET Core Web API (EF Core Code-First, SQL Server, cookie session auth, Swagger)

Repository contents

Path	Purpose
docs/erd.png, docs/erd.pdf	Entity Relationship Diagram (6 entities)
docs/schema.sql	SQL Server script: tables, constraints, seed data
docs/api_endpoint_plan.md	Part 1 API endpoint plan (the specification for Part 2)
Controllers/, DTOs/, Models/, Data/, Program.cs	Part 2 API source
.github/workflows/	GitHub Actions workflows (see below)

Data model (6 entities)

1. **Users** - Organisers and Participants; the role is stored on the user.
2. **UserProfiles** - 1:1 with Users; phone number, date of birth, bio.
3. **Events** - 1:M from Users (Organiser); name, description, date, location, distance, type (Run, Walk, Cycle).
4. **Categories** - 1:M from Events; each event defines its own age/distance categories.
5. **Enrolments** - links a Participant, an Event and the chosen Category. A participant can enrol in an event only once (unique on ParticipantId + EventId).
6. **Results** - 1:1 with Enrolments; finish time and finishing position.

Part 2 - API status

Area	Status
Project setup (ASP.NET Core, EF Core Code-First, SQL Server)	Done
Authentication: register, login, logout (cookie session, hashed passwords)	Done
User Profile: GET/PUT /api/users/me	Done

Area	Status
Swagger UI at /swagger	Done
EF Core seed data matching schema.sql	Done
Events (Organiser manages, both roles view)	Not started
Categories	Not started
Event Enrolments	Not started
Results	Not started
Unit tests	Not started
CI workflow running the tests	Workflow written, manual trigger only until the test project exists

The implemented endpoints match docs/api_endpoint_plan.md (routes, roles, request bodies and status codes). The remaining endpoints in the plan are still to be built.

Authentication and roles

- Users register with their details and choose a role (Organiser or Participant).
- Passwords are hashed with ASP.NET Core's PasswordHasher (PBKDF2); the original password is never stored.
- Login creates a cookie-based session that carries the user's id and role. Swagger's "Try it out" works without extra setup because the browser sends the session cookie automatically.
- Non-public endpoints return 401 for unauthenticated requests and 403 when the role is wrong.
- Profile endpoints use the id from the session, never a route id, so a user can only read or edit their own profile.

Running the API locally

1. **Packages.** The project needs .NET 8 or later and these packages:
2. dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.10
3. dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.10
4. dotnet add package Microsoft.Extensions.Identity.Core --version 8.0.10
5. dotnet add package Swashbuckle.AspNetCore --version 6.7.3
6. dotnet add package Microsoft.OpenApi --version 1.6.22

Do **not** have Microsoft.AspNetCore.OpenApi installed. It pulls in Microsoft.OpenApi 2.x, which breaks Swashbuckle 6.7.3 (Swagger returns a 500 on /swagger/v1/swagger.json).

7. **Connection string** in appsettings.json:

8. "ConnectionStrings": {
9. "DefaultConnection": "Server=localhost;Database=RaceDayDB_Api;Trusted_Connection=True;TrustServerCertificate=True;"
10. }

Change Server= to match your SQL Server instance. Use a database name different from the one docs/schema.sql creates (RaceDayDB); both create the same tables, so running both against one database fails.

11. **Create the database and run:**
12. dotnet tool install --global dotnet-ef
13. dotnet ef migrations add InitialCreate
14. dotnet ef database update
15. dotnet run

Open https://localhost:<port>/swagger.

Seeded development accounts

All four use the password Password123!.

Role	Email
Organiser	sarah.nkosi@raceday.co.za
Organiser	david.botha@raceday.co.za
Participant	lindiwe.dube@example.com
Participant	michael.reddy@example.com

Differences between schema.sql and the Code-First schema

The EF Core model mirrors docs/schema.sql: the same six tables, keys, unique, check and default constraints, and the same constraint and index names. The deliberate differences are:

1. **Seeded password hashes.** schema.sql uses placeholder text for PasswordHash, which can never match a real password. The EF seed uses a real ASP.NET Core Identity PBKDF2 hash so the seeded users can log in.
2. **Seeded timestamps.** CreatedAt, EnrolmentDate and CapturedAt are fixed values in the EF seed, because EF seed data must be deterministic. schema.sql uses GETDATE().
3. **Role values.** The seed rows and constraints are identical. In the C# enum, Participant is listed first (value 0) so that the database default of 'Participant' does not overwrite Organisers. This does not change the stored text.

GitHub Actions

- validate-docs.yml runs docs/schema.sql against a clean SQL Server container and checks that all six tables and the seed data exist. It runs on changes under /docs.
- ci.yml builds the API and runs the unit tests against a SQL Server container. It is set to manual trigger (workflow_dispatch) until the Part 2 test project exists; then uncomment its push and pull_request triggers and set the project paths in the migrations step.